COS214 Practical Assignment 1

Caleb Jennings - u25173805 Chloe Larsen - u25004141

Task 1

1.5) Factory method is the better method as it adheres to the open-closed principle (open for extension and closed for modification).

With Factory Method:

- The abstract class *ConnectorFactory*, which is the creator class, declares the pure virtual factory method of *Connector* createConnector()*
- Each concrete factory (*CsvFactory*, *PostgresFactory*, *RestApiFactory*) implements its own version of the *createConnector()* function to return its own version of the connector type.
- Adding a new data source only requires a new concrete factory to be created and implemented with no modification to completed code.

With if/else chain

- A single factory class would contain an if/else chain that will return the connector for whatever value is in the true section of the train.
- When adding a new factory the completed code would have to be modified in order for the factory to be implemented.

Therefore by implementing with the factory method the classes are open for extension (new factories can easily be added) but closed to modification (changes cannot be made to the implemented files), perfectly following the open-closed principle.

1.6)

- *Connector* -> **Creator** (creates product which the client uses)
- *CsvConnector* -> **ConcreteCreators** (create specific concrete product)
- *ConnectorFactory* -> **Product** (is always created by **Creator**)
- *CsvFactory* -> **ConcreteCreators** (create specific concrete product)

Task 2

2.5) Shallow Copy

Creates a new object from the data from an existing object, however it duplicates the top level structure of the existing object. Therefore it copies over the memory addresses or references from the existing object, meaning if the existing object is modified or deleted then the copied object will also be modified or deleted.

Deep Copy

Creates a new object from the data from an existing object, it will make independent copies of every part of the existing object therefore completely separating it from the existing object. Therefore no change to the existing object will reflect on the copied object.

The *clone()* function uses **deep copy** to create the new object and it must do this to ensure there is independence between the two objects. The clone must be independent of the existing object to ensure that modifications to either object affects only the selected object.

2.6) The registry returns clones rather than stored pointers in order to prevent multiple pipelines from sharing and potentially modifying the same transformation object. This encapsulation ensures that there is isolation between different pipeline objects.

Task 3

3.5) Making *run()* a non-virtual function ensures that the algorithm structure remains consistent and cannot be modified by subclasses. If a subclass could override the *run()* function then it could mess up the algorithm structure for that subclass.

3.6)

- *run()* -> Template Method (defines the skeleton algorithm)
- *connect()* -> Concrete Step (shared implementation)
- *extract()* -> Primitive Operation (pure virtual)
- *transform()* -> Concrete Step (shared implementation)
- *load()* -> Primitive Operation (pure virtual)

Factory Method provides a way to create source connectors without knowing concrete types, allowing *pipelines* to work with different data sources. **Template Method** allows the pipeline's subclasses to have customisable steps but ensures execution order. They do not overlap because the factory method is responsible for the creation of the objects and the template method deals with algorithm structure.

Task 4

4.4) The **narrow interface** exposes the getters (*getStage()*, *getRecords()* used by the **Caretaker** (*CheckpointManager*)) to store and restore state. The **wide interface** includes setters and other methods that could modify the **Momento's** state, which is only accessible by the **Originator** (*Pipeline*). The **Caretaker** will only use the narrow interface to ensure encapsulation.

4.5) When a *run()* fails during *load()*, the recovery process works by:

1) The system calls *undo()* on *CheckpointManager* to retrieve the last saved checkpoint
2) The *Pipeline's restore()* method is then called by *Checkpoint* to set the stage back to the last completed stage
3) The system then checks the stage and resumes execution from the *load()* function without re-executing the previous stage.
4) The *createCheckpoint()* and *save()* functions are used periodically during normal operation to ensure that there are captured safe points.

4.6)

- *Pipeline* -> **Originator** (creates and stores **Mementos**)
- *RunCheckpoint* -> **Mementos** (stores the internal state of the **Originator**)
- *CheckpointManager* -> **Caretaker** (stores and manages **Mementos**)